

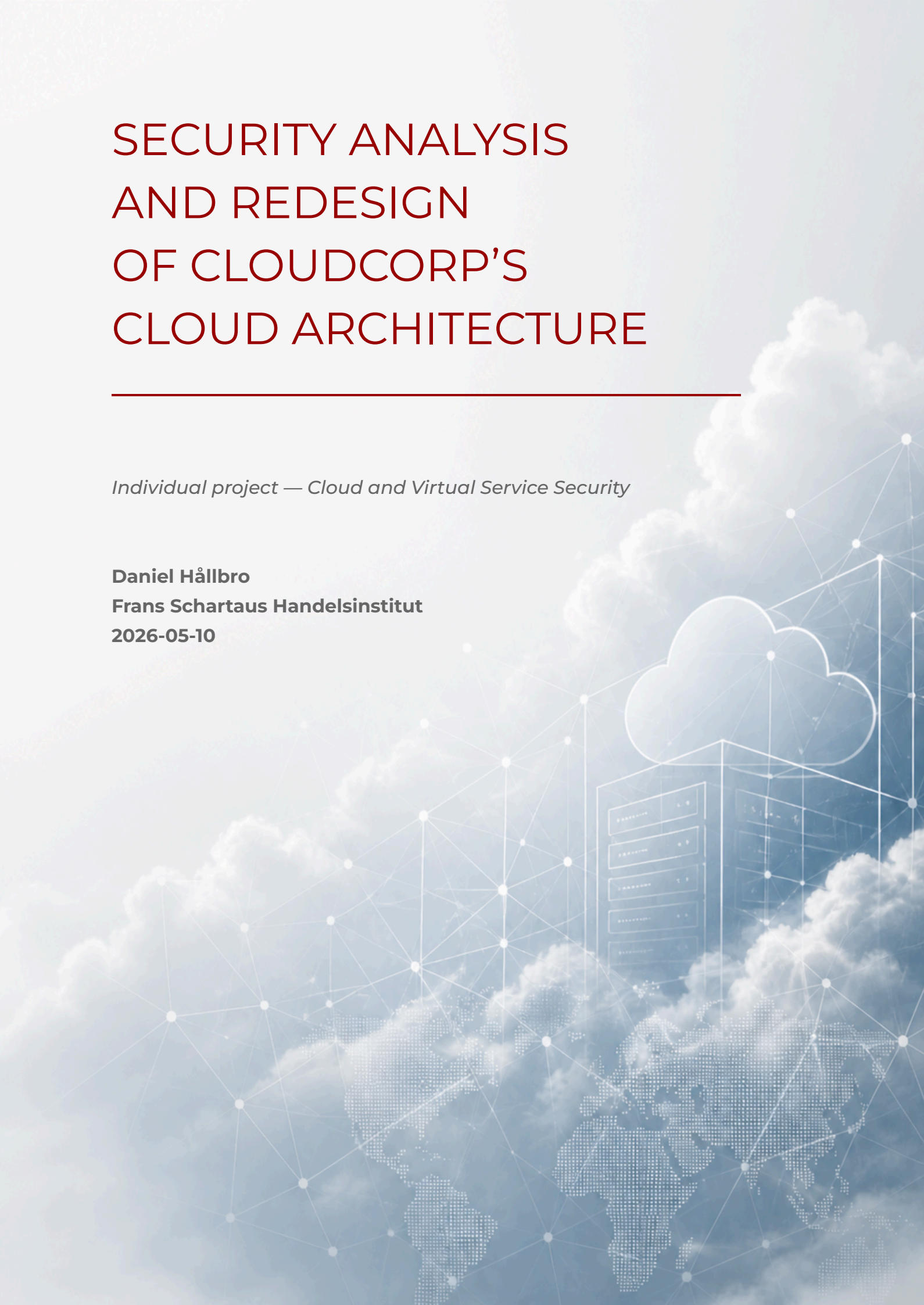
SECURITY ANALYSIS AND REDESIGN OF CLOUDCORP'S CLOUD ARCHITECTURE

Individual project — Cloud and Virtual Service Security

Daniel Hållbro

Frans Schartaus Handelsinstitut

2026-05-10



Executive summary

Background

CloudCorp recently suffered a data breach where customer data was exposed through a misconfigured cloud storage bucket. The breach required no sophisticated attack — basic settings were left open to the internet. Without logging in place, the full scope of what was accessed cannot be determined retrospectively, which directly affects CloudCorp's ability to meet GDPR's 72-hour breach notification requirement and complicates customer communication.

Approach

This report analyses what went wrong, redesigns the cloud environment around modern security principles, and weighs each design choice against the risk it addresses and what it costs. Verified cost figures are documented in Appendix A.

Results

The redesigned architecture protects customer data against accidental exposure, regional outages, and most categories of insider misuse. Every action in the environment is now logged and auditable, restoring CloudCorp's ability to detect and report incidents in line with GDPR. Multiple independent controls protect each critical asset, so a single misconfiguration can no longer expose customer data the way it did in the original incident. The total operating cost is approximately **\$335/month (~3,500 SEK)** for the production environment.

Recommended next steps

- **Approve the redesigned architecture** for deployment.
- **Approve the operational budget** documented in Appendix A.
- **Invest in cloud expertise** — either hire a cloud security engineer or fund AWS certifications for existing staff. The architecture's effectiveness depends on competent operation.
- **Schedule a 6-month review** to right-size the design based on observed usage.

Table of contents

Executive summary.....	2
Table of contents.....	3
Introduction and scope.....	4
Part 1 – Incident analysis.....	4
1.1 What was the problem?.....	4
1.2 Which security principles were broken?.....	5
1.3 What were the consequences?.....	6
1.4 Mapping to the CIA triad.....	6
Part 2 — New architecture.....	8
2.1 Component walkthrough.....	9
2.2 Design principles.....	10
2.3 Network design.....	10
2.4 Compute and availability.....	11
2.5 Data storage.....	11
2.6 Identity and access.....	12
2.7 Edge and perimeter protection.....	12
2.8 Observability.....	13
2.9 Summary.....	13
Part 3 — Implementation.....	14
3.1 Network.....	14
3.2 Security Groups.....	15
3.3 Storage and encryption.....	16
3.4 Identity and access.....	17
3.5 Compute.....	18
3.6 Edge and perimeter.....	19
3.7 Observability.....	20
Part 4 — Network and secure design.....	22
4.1 Subnets and routing.....	22
4.2 Security Groups.....	22
4.3 Attack surface.....	23
Part 5 — Security analysis.....	24
5.1 Approach.....	24
5.2 Five identified risks.....	24
5.3 Improvement roadmap.....	26
Part 6 — Cost and cloud service models.....	27
6.1 IaaS, PaaS and SaaS.....	27
6.2 Where the costs come from.....	27
6.3 Cost risks.....	28
6.4 Cost optimisations.....	29
Reflection.....	30

Introduction and scope

CloudCorp sells a SaaS-service to their customers, but they themselves use IaaS-services from AWS to power their product. This report will not focus on the SaaS delivery to the customer, but the underlying IaaS-layer where CloudCorp has full operational and security responsibility. That's where the incident occurred and where the redesign needs to take place.

Based on the scenario describing a single EC2 instance and a single S3 bucket holding all customer data, this report assumes CloudCorp operates a multi-tenant architecture where all customers share the same underlying infrastructure. A single-tenant model (per-customer environments) would result in a fundamentally different design.

The scope follows the AWS Shared Responsibility Model: AWS is responsible for the security *of* the cloud (physical infrastructure, hardware etc), while CloudCorp is responsible for security *in* the cloud (network configuration, IAM, encryption, logging etc). Since CloudCorp is also a data processor under GDPR for its customers' data, the responsibility is not only technical but also legally binding.

Throughout the report I lean on the AWS Well-architected Framework (Security Pillar) together with classic security principles like *least privilege*, *defence in depth*, *attack surface reduction* and *zero trust*. Each design choice is motivated both in terms of what it protects and what the consequence is if the control fails.

Part 1 – Incident analysis

1.1 What was the problem?

CloudCorp's environment was broken on essentially every layer. This is not a matter of isolated mistakes, but a pattern where default settings and convenience echoes throughout the topology in favour of sound security thinking. I have decided to split the incident up in five areas.

1.1.1 Network

The entire environment lived in a single public subnet with no segmentation, meaning the backend and data layer were exposed to the internet just like the frontend. Security Groups allowed traffic on port 22 (SSH) and 80 (HTTP) from 0.0.0.0/0 — anyone on the internet. There was no bastion host, no load balancer and no WAF in front of the application.

1.1.2 Storage

The S3 bucket holding customer data had Public Read enabled, no encryption and no bucket policies. Public S3 buckets are not hard to find — there are bots that continuously scan AWS for exactly this kind of thing, and Google indexes the contents on top of that if it's not blocked. This is how the incident started.

1.1.3 IAM

A single IAM user had full S3 access. If that user's credentials leaked (phishing, github commits, EC2 compromise), an attacker would directly get read, write and delete privileges across all customer data. There was no role-based access, no MFA, and no separation between application and administrator.

1.1.4 Logging and monitoring

There was no CloudTrail, no CloudWatch — no audit trail of API calls and no metrics or alarms. CloudCorp therefore has no way of telling what the attacker did, what data was actually taken, or whether a backdoor was left behind.

1.1.5 Cost monitoring

No budget alerts means cost-based attacks (cryptomining on hijacked compute, EDoS attacks designed to exhaust resources) go unnoticed until the next AWS invoice. Cost monitoring is a security control, not just a finance one.

1.2 Which security principles were broken?

Four core principles can be linked to the flaws above. The point is not to repeat them, but to highlight *why* each one matters.

The whole reason *least privilege* exists is that an environment is only as secure as its weakest credential. CloudCorp gave one IAM user full access to all customer data, which means a single phishing email or accidental github commit could have been enough to lose everything. The same pattern shows up at the network layer — SSH open to 0.0.0.0/0 means every bot on the internet can attempt to log in.

Defence in depth assumes that any single control will eventually fail, which is why you stack several. Here there was nothing else to slow the attacker down once the bucket was public — no encryption, no bucket policy, no alarm. The whole posture rested on one setting being correct.

Then there's *attack surface reduction*, which is the principle of not exposing what doesn't need to be exposed. Backend and data layers should sit behind something — a load

balancer, a CDN, anything — and not accept traffic directly from the internet. In CloudCorp's setup, everything was reachable.

Finally, *zero trust* assumes nothing on the inside is automatically safe. The challenge here is that there isn't really an "inside" — everything sits in one public subnet with no segmentation, so getting in at one point gives access to all of it.

1.3 What were the consequences?

The direct consequence was that customer data was exfiltrated from the S3 bucket. The attacker did not even need to exploit a vulnerability in the classic sense – the gross misconfiguration *was* the attack vector. A web browser and the right URL is all it takes.

The legal side is also a problem. GDPR requires that a personal data breach is reported within 72 hours, but in order to report something you need to know what happened. Since there are no logs, it's essentially impossible for CloudCorp to answer which data was read, by whom, or when. Not being able to directly inform customers exactly what has happened may be required by law in cases where data leak is extensive.

Reputational damage is harder to quantify but is often the most expensive consequence in the long run. For a SaaS company in this day and age, the customers' trust in data handling is part of the product itself. A public data leak affects both existing contracts and future endeavours alike.

As there was no logging present in combination with a serious SSH misconfiguration, there is no way of telling how compromised the EC2-instance is. We could be looking at backdoors, compromised source code or active listeners placed in the environment.

1.4 Mapping to the CIA triad

In this section I will map flaws onto the three components of the CIA triad.

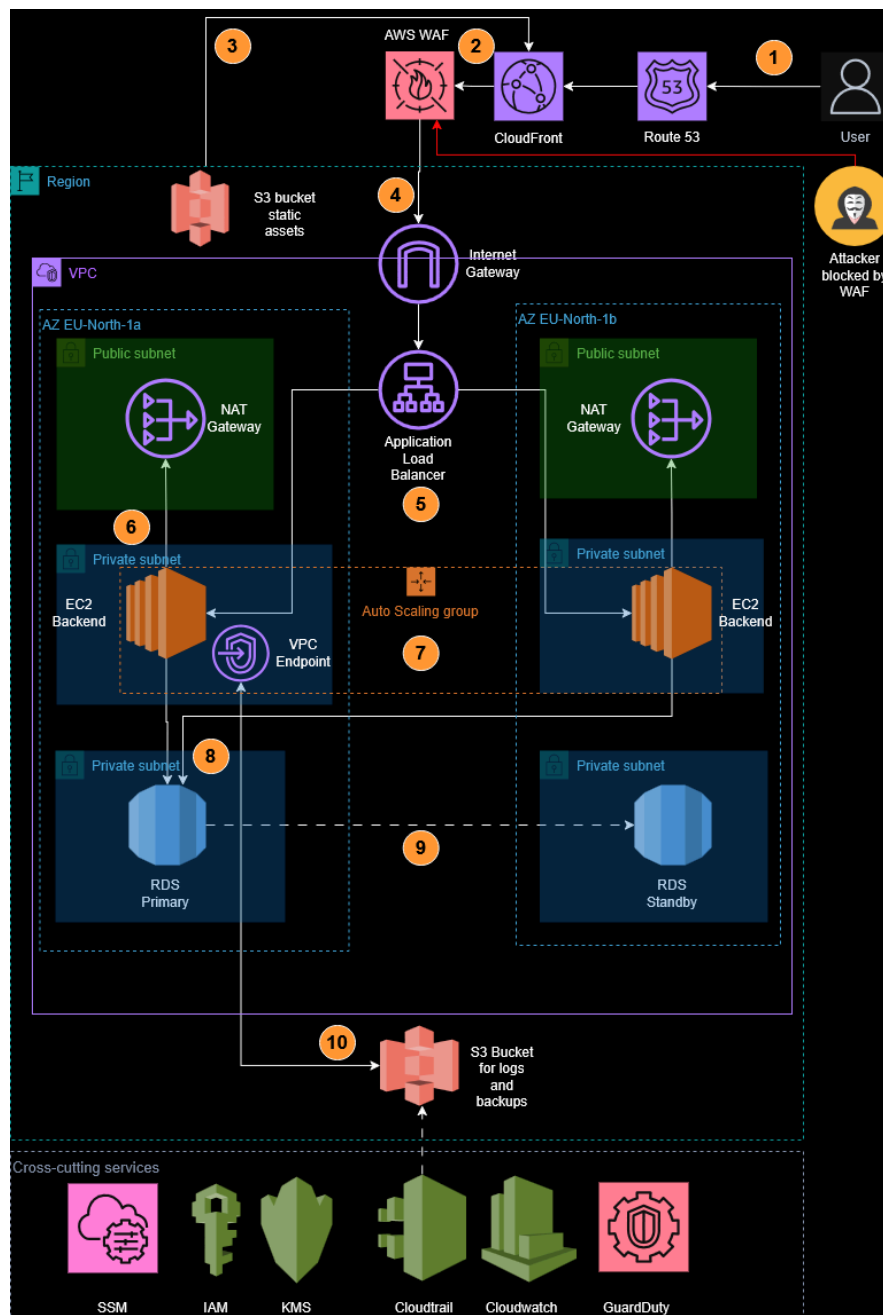
Confidentiality failed at the bucket level — public read with no encryption meant anyone with the URL could download the data. It also failed at IAM, since a single set of credentials guarded everything.

Integrity is uncertain both retrospectively and going forward. With full write privileges and no versioning, the attacker could have modified or replaced objects without trace. The same applies to the EC2 side, where code or configuration may have been tampered with.

Availability suffered from the architecture itself. A single EC2 instance with no load balancer, auto scaling or multi-AZ redundancy means any disruption — attack, operational error, or AWS zone incident — takes the whole service down.

Part 2 — New architecture

The diagram below shows the redesigned architecture, with numbered annotations tracing the path of a typical request. The table maps each number to its role, and the sections that follow explain **why** each piece is there. Cost is touched on lightly here to justify individual choices; the full breakdown lives in Part 6.



2.1 Component walkthrough

#	Component	What happens
1	Route 53	DNS resolves cloudcorp.com via Alias records pointing to CloudFront.
2	CloudFront + WAF	CloudFront caches and serves traffic from the nearest edge location. AWS WAF inspects every request and blocks known attack patterns before they reach the VPC.
3	S3 (static assets)	CloudFront serves static files (CSS, JS, images) directly from a private bucket via Origin Access Control.
4	Internet Gateway	Dynamic traffic enters the VPC through the IGW after passing the edge filters.
5	Application Load Balancer	The ALB receives traffic and distributes it across healthy EC2 instances in both AZs. TLS is terminated at CloudFront; the ALB handles internal routing.
6	EC2 Backend	Application servers in the private app subnet handle requests. No public IPs, no SSH.
7	Auto Scaling Group	Scales EC2 capacity automatically between 2 and 4 instances based on load and replaces unhealthy instances.
8	RDS Primary	Structured customer data is stored in MySQL. Reachable only from the EC2 security group.
9	RDS Standby	Synchronous replication to a standby in the other AZ. Automatic failover within 60-120 seconds.
10	S3 (logs/backups)	EC2 writes logs and backups to S3 through a VPC Endpoint without traversing the public internet.
△	Attacker	Attempts from the internet are blocked at AWS WAF before reaching the VPC. Defense in depth means subsequent layers (ALB, security groups, IAM, encryption) act as backups even if WAF fails.

2.2 Design principles

The new architecture is based on three principles: **defence in depth, least privilege, and minimal attack surface**. Each choice is motivated both in terms of what it protects against and what it costs relative to the risk it addresses. Security in the cloud is not about enabling everything that exists — it's about enabling the *right* things given the value being protected. Since CloudCorp handles personal data under GDPR and has already suffered an incident, the cost of doing security right is easier to motivate than it would be for a brand new project with no track record.

Defence in depth is the principle that runs through the whole architecture, so it's worth grounding what it actually means here. The network is built in layers, each doing one specific job, and breaking through one does not give access to the next. Traffic from the internet hits CloudFront and WAF first → then the Internet Gateway → then the ALB → then the Security Group around EC2 → and only then the application itself. The database sits a layer further in, reachable only from EC2 on a single port.

Each of these layers exists because the layer in front of it *might fail*. WAF filters known attack patterns, but a brand new exploit might slip through. A Security Group blocks unexpected ports, but a misconfiguration might open one. The next layer assumes the previous one failed and adds its own check. This is the direct counterweight to the original incident, where one misconfigured bucket was enough to expose everything — there was nothing behind it.

2.3 Network design

The architecture is built in a VPC with subnets distributed across **two Availability Zones** for redundancy, with three subnets per AZ: public, private app, and private data. The detailed motivation for this segmentation lives in Part 4.2.

Multi-AZ is a deliberate choice even though it costs more. AWS has historically had AZ-level incidents roughly once a year, and without Multi-AZ every such incident means total service loss until manual recovery. The premium of roughly \$65/month is justifiable for a production environment where availability is part of the CIA triad we set out to protect.

2.4 Compute and availability

The application servers run on **EC2 instances in an Auto Scaling Group** across the two AZs in the private app subnets. An **Application Load Balancer (ALB)** in the public subnet receives all incoming traffic and distributes it to the instances. CloudFront handles TLS termination toward clients; the ALB manages internal routing and distributes traffic to EC2 instances.

Auto Scaling addresses both availability and a concrete *integrity risk*: if an instance is compromised it can be terminated and automatically replaced with a clean AMI, meaning that compromises do not survive long in the environment.

2.5 Data storage

Data storage is split across two separate services depending on data type.

Amazon RDS (PostgreSQL or MySQL) runs in Multi-AZ configuration in the private data subnets. This is where structured customer data lives: account information, transactions, application state. RDS was chosen over a self-managed database on EC2 for the same reason most managed services are: AWS handles patching, backup and failover. It is more expensive per hour than self-hosting but cheaper overall when staff cost and misconfiguration risk are factored in. For an organization that the scenario describes as "lacking deep security expertise", managed services are an especially strong choice.

Amazon S3 is used for object storage that does not belong in a relational database: customer-uploaded files, RDS backups, and log archives from CloudTrail and the ALB. A separate bucket holds the application's static assets and is served through CloudFront. All buckets have **Block Public Access** enabled at the account level, **encryption with KMS** at rest, and **bucket policies** requiring HTTPS for all access (**aws:SecureTransport**). **Versioning** is enabled to protect against accidental or malicious deletion. The data bucket additionally has MFA Delete, since being able to silently delete forensic evidence or recovery data is exactly the kind of thing an attacker would want. These are direct countermeasures to exactly the scenario that hit CloudCorp: even if a bucket were accidentally made public in the future, the Block Public Access setting would stop access at the account level.

The data bucket additionally has **Cross-Region Replication** (CRR) enabled to a second AWS region. This protects against a regional AWS outage, ransomware-style data destruction in the primary region, and accidental bulk deletion. Full active-passive multi-region failover for the application itself is deferred (see Part 5), but replicating backups is cheap (a few dollars per month) and removes a single-region risk for the data that matters most.

The EC2 instances reach S3 via a **VPC Gateway Endpoint** instead of over the internet. This means that the S3 traffic never leaves the AWS network, which both eliminates an exfiltration path and reduces NAT Gateway costs.

2.6 Identity and access

IAM is rebuilt from the ground up. The previous user with full S3 access is replaced by two separate mechanisms.

The application uses an **IAM role** attached to the EC2 instance profile, with policies allowing only the specific S3 prefixes and RDS connections the application needs. No static access keys are stored in environment variables or on disk, which eliminates an entire category of credential leaks (AWS regularly reports leaked keys on GitHub that attackers find within minutes).

For administrative access, **IAM roles with MFA requirements** and least-privilege policies are used, separated by area of responsibility (database admin, network admin, etc). Login to EC2 instances happens through **AWS Systems Manager Session Manager**, which replaces both SSH and the bastion host. Session Manager requires no open ports, no key management, and logs every session to CloudTrail. It is both more secure and cheaper than a traditional bastion host.

2.7 Edge and perimeter protection

All external traffic reaches the environment through several filtering layers before hitting the ALB.

Amazon CloudFront acts as a CDN, distributing both static content from S3 and dynamic content from the ALB. CloudFront automatically provides **AWS Shield Standard** against DDoS, terminates TLS close to the user, and hides the ALB's origin IP. CloudFront's permanent free tier (1 TB transfer and 10M requests/month) absorbs most of the cost — for CloudCorp's modeled 1.2 TB and 15M requests, only the excess is billable, keeping CloudFront at around \$24/month while still providing global edge presence and DDoS absorption capacity that would otherwise require expensive third-party services.

AWS WAF is attached to CloudFront and blocks the most common attacks (SQL injection, XSS, known malicious IP ranges, geo-blocking where relevant). For a few dollars per month, a large portion of automated attacks are stopped before reaching the application — defense in depth at its simplest.

Route 53 is used for DNS, with Alias records pointing to CloudFront. An ACM certificate is configured on both CloudFront and the ALB, enabling end-to-end HTTPS from client to application. Health checks and active-passive multi-region failover were considered but deferred (see Part 5).

The full attack-surface analysis — what these layers achieve and what remains exposed — is in Part 4.5.

2.8 Observability

CloudTrail is enabled with an organization-wide trail that logs all API calls to a dedicated S3 bucket with encryption and log file validation enabled (so that logs cannot be tampered with undetected). **CloudWatch** collects metrics and logs from the

ALB, EC2, RDS and WAF, with alarms for anomalous patterns (sudden traffic spikes, high error rates, failed logins).

Amazon GuardDuty is enabled as threat detection. It is a managed service that analyzes CloudTrail, VPC Flow Logs and DNS logs against known threat patterns. Cost figures are documented in Appendix A. The value is that CloudCorp gets automatic warnings if for example an EC2 instance starts communicating with known C2 servers or an IAM user behaves anomalously, addressing the previously total lack of detection capability.

2.9 Summary

The architecture is built around the idea that no single control should carry the whole weight of security. Traffic is filtered at the edge, segmented inside the VPC, restricted by IAM, encrypted at rest, and observed at every step. If one layer fails — and at some point one will — the others are still there. That is the direct counterweight to the original incident, where one misconfigured bucket was enough to expose everything.

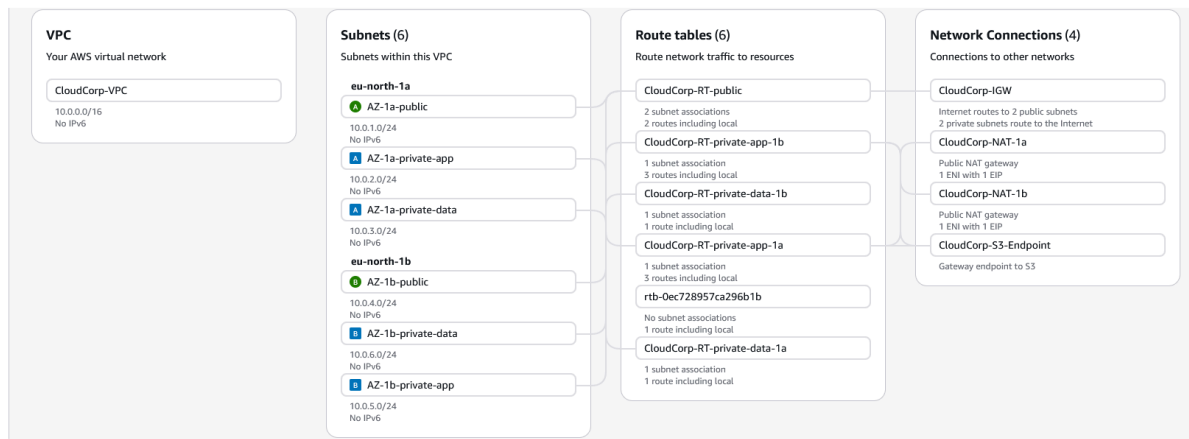
Part 3 — Implementation

This section documents the actual deployment of the architecture described in Part 2, carried out in AWS eu-north-1 (Stockholm). Screenshots are included for the most significant configuration points. A few limitations apply to the demo environment that would not exist in production — these are noted where relevant.

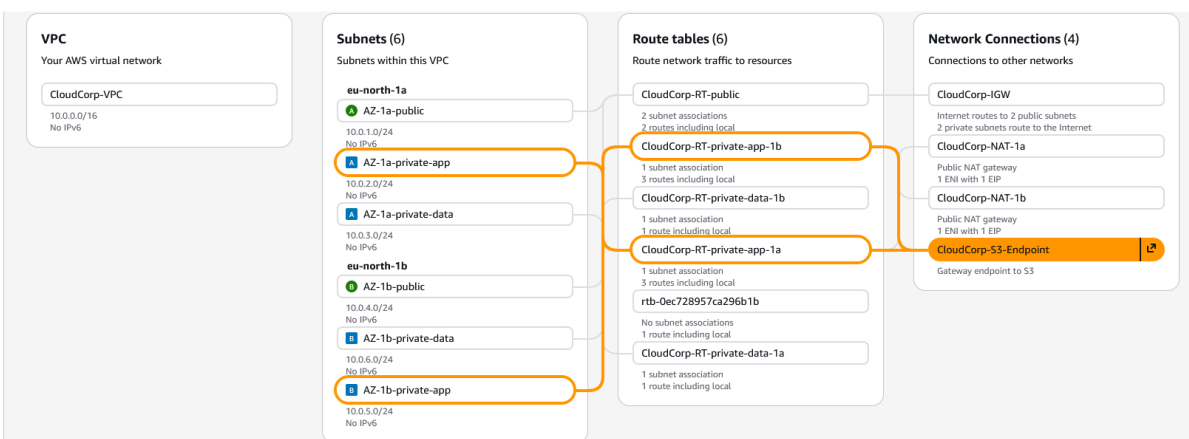
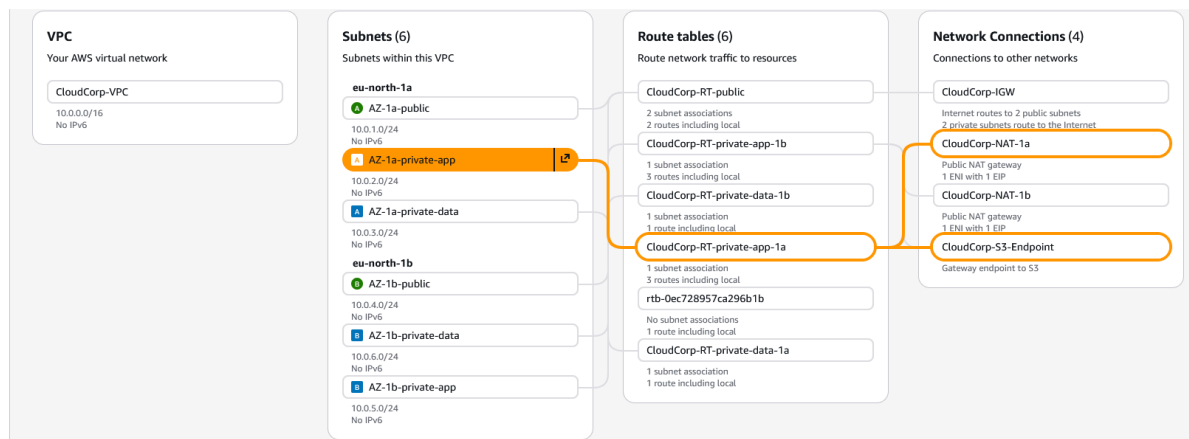
3.1 Network

The VPC was configured with six subnets across two Availability Zones, three per AZ: one public, one private app, and one private data. Five route tables enforce the segmentation: the public route table routes outbound traffic through the Internet Gateway, the private app route tables route through their respective NAT Gateways (one per AZ to avoid a single point of failure), and the private data route tables have no default route at all — RDS instances there cannot initiate or receive internet traffic. An S3 VPC Gateway Endpoint was attached to both private app route tables, keeping S3 traffic off the NAT Gateway entirely.

Daniel Hållbro @ CS25 IT- and Cybersecurity Specialist
Security analysis and redesign of CloudCorp's cloud architecture
Cloud and Virtual Service Security (30p)



vpc-0197e2b0fc256b3c9 / CloudCorp-VPC



3.2 Security Groups

Three Security Groups enforce a strict chain: the ALB accepts inbound HTTPS (443) only from CloudFront's managed prefix list ([com.amazonaws.global.cloudfront.origin-facing](https://com.amazonaws.global/cloudfront.origin-facing)), EC2 accepts inbound HTTPS only from the ALB Security Group, and RDS accepts inbound MySQL (3306) only from the EC2 Security Group. Nothing else can reach any component directly.

Daniel Hållbro @ CS25 IT- and Cybersecurity Specialist
Security analysis and redesign of CloudCorp's cloud architecture
Cloud and Virtual Service Security (30p)

Outbound rules are equally restricted. ALB forwards only to EC2 on 443. EC2 is allowed outbound HTTPS for OS updates and AWS service communication (SSM, CloudWatch), and MySQL to the RDS Security Group. RDS has no outbound rules. SSM Session Manager requires no inbound rules — EC2 initiates the connection outbound to AWS endpoints, which is why there is no SSH port open anywhere.

sg-053eb97e30c9e61b6 - CloudCorp-SG-EC2

Details | **Inbound rules** | Outbound rules | Sharing | VPC associations | Related resources - new | Tags

Inbound rules (1)

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source
-	sgr-0329d6be063e35db2	-	HTTPS	TCP	443	sg-0bc6be4dbefe359b6 / CloudCorp-SG-ALB

sg-07dc96730385e1ae4 - CloudCorp-SG-RDS

Details | **Inbound rules** | Outbound rules | Sharing | VPC associations | Related resources - new | Tags

Inbound rules (1)

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
-	sgr-0cb2ff41124a42173	-	MySQL/Aurora	TCP	3306	sg-053eb97e30c9e61b...	-

Name	Security group ID	Security group name	VPC ID	Description	Owner
-	sg-07dc96730385e1ae4	CloudCorp-SG-RDS	vpc-0197e2b0fc256b3c9	RDS Security Group	811261690901
-	sg-0bc6be4dbefe359b6	CloudCorp-SG-ALB	vpc-0197e2b0fc256b3c9	Load Balancer Security Group	811261690901
-	sg-053eb97e30c9e61b6	CloudCorp-SG-EC2	vpc-0197e2b0fc256b3c9	EC2 Security Group	811261690901

sg-0bc6be4dbefe359b6 - CloudCorp-SG-ALB

Details | **Inbound rules** | Outbound rules | Sharing | VPC associations | Related resources - new | Tags

Inbound rules (1)

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source
-	sgr-0a57719a70792e1fe	-	HTTPS	TCP	443	pl-fab65393 (com.amazonaws.global.cloudfront.origin-facing)

3.3 Storage and encryption

The most important screenshot in this section directly addresses the root cause of the original incident: account-level Block Public Access, enabled across all four settings. A bucket misconfiguration like the one that caused the breach cannot happen in this environment — the account-level setting overrides any bucket-level configuration.

Daniel Hållbro @ CS25 IT- and Cybersecurity Specialist
Security analysis and redesign of CloudCorp's cloud architecture
Cloud and Virtual Service Security (30p)

Block Public Access settings for this account successfully updated.

Account and organization settings

Block Public Access settings for this account

Edit

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies or all. In order to ensure that public access to all your S3 buckets and objects is blocked, turn on Block all public access. These settings apply account-wide for all current and future buckets and access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to your buckets or objects, you can customize the individual settings below to suit your specific storage use cases. [Learn more](#)

Block all public access

☒ On

- ☒ Block public access to buckets and objects granted through **new** access control lists (ACLs)
- ☒ Block public access to buckets and objects granted through **any** access control lists (ACLs)
- ☒ Block public access to buckets and objects granted through **new** public bucket or access point policies
- ☒ Block public and cross-account access to buckets and objects through **any** public bucket or access point policies

Two S3 buckets were created in Stockholm: **cloudcorp-data** for customer files, logs and backups, and **cloudcorp-static-assets** for web assets. Both have versioning enabled on the data bucket and KMS encryption with a customer-managed key (CloudCorp-KMS-Stockholm). Cross-Region Replication is configured on the data bucket, replicating asynchronously to **cloudcorp-data-crr** in Frankfurt using a separate KMS key (CloudCorp-KMS-Frankfurt).

Buckets

General purpose buckets All AWS Regions Directory buckets

General purpose buckets (3)

Info

Refresh Copy ARN Empty Delete Create bucket

Buckets are containers for data stored in S3.

Find buckets by name

Name	AWS Region	Creation date
cloudcorp-data-811261690901	Europe (Stockholm) eu-north-1	May 7, 2026, 09:17:49 (UTC+02:00)
cloudcorp-data-crr-811261690901	Europe (Frankfurt) eu-central-1	May 7, 2026, 09:34:39 (UTC+02:00)
cloudcorp-static-assets-811261690901	Europe (Stockholm) eu-north-1	May 7, 2026, 09:18:28 (UTC+02:00)

Amazon S3 > Buckets > cloudcorp-data-811261690901 > Replication rules

Replication configuration successfully updated

Replication rules

Replication configuration settings

Replication rules (1)

Replication rule name	Status	Destination bucket	Destination Region	Priority	Scope	Storage class	Replica owner	Replication Time Control	KMS-encrypted objects (SSE-KMS or DSSE-KMS)	Replica modification sync
CloudCorp-CRR-Rule	Enabled	s3://cloudcorp-data-crr-811261690901	Europe (Frankfurt) eu-central-1	0	Entire bucket	Same as source	Same as source	Disabled	Replicate	Disabled

3.4 Identity and access

The EC2 instance role (CloudCorp-EC2-Role) follows least privilege: it can read and write to the data bucket, read from the static assets bucket, and use the KMS key for those operations — nothing else. SSM and CloudWatch agent access come from AWS managed policies (AmazonSSMManagedInstanceCore and CloudWatchAgentServerPolicy) rather than custom ones, keeping the custom policy focused on S3 and KMS only.

Two administrative roles separate responsibilities: CloudCorp-InfraAdmin-Role covers network, compute and storage configuration, while CloudCorp-SecurityAdmin-Role covers CloudTrail, GuardDuty and CloudWatch. Neither role has access to the other's domain, and neither can read customer data directly — that access belongs exclusively to the EC2 instance role. Keeping infrastructure administration separate from security monitoring means that a compromised InfraAdmin credential cannot be used to disable logging or tamper with audit trails — and vice versa.

Review and create Info
Review the permissions, specify details, and tags.

Policy details
Policy name
Enter a meaningful name to identify this policy.

Maximum 128 characters. Use alphanumeric and "+, -, @, _" characters.
Description - optional
Add a short explanation for this policy.

Maximum 1,000 characters. Use alphanumeric and "+, -, @, _" characters.

Permissions defined in this policy Info Edit
Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), attach a policy to it

Allow (5 of 469 services) Show remaining 464 services

Service	Access level	Resource	Request condition
CloudTrail	Full access	All resources	None
CloudWatch	Full access	All resources	None
CloudWatch Logs	Full access	All resources	None
GuardDuty	Full access	All resources	None
KMS	Full; List Limited: Read	All resources	None

Role CloudCorp-Policy-EC2-S3 created. View role ×

CloudCorp-Policy-EC2-S3 Info Delete
Allows EC2 instances to call AWS services on your behalf.

Summary Edit

Creation date
May 07, 2026, 09:49 (UTC+02:00)

Last activity
-

ARN
 `arn:aws:iam::811261690901:role/CloudCorp-Policy-EC2-S3`

Instance profile ARN
 `arn:aws:iam::811261690901:instance-profile/CloudCorp-Policy-EC2-S3`

Maximum session duration
1 hour

Permissions | Trust relationships | Tags | Last Accessed | Revoke sessions

Permissions policies (3) Info Simulate Remove Add permissions
You can attach up to 10 managed policies.
 Filter by Type All types

☐	Policy name	Type	Attached entities
☐	AmazonSSMManagedInstanceCore	AWS managed	2
☐	CloudCorp-Policy-EC2-S3	Customer managed	1
☐	CloudWatchAgentServerPolicy	AWS managed	1

3.5 Compute

EC2 instances run in private app subnets with no public IPs, launched by an Auto Scaling Group configured for minimum 2, desired 2, maximum 4 instances across both AZs. The Application Load Balancer sits in the public subnets and forwards HTTPS to the target group. HTTP traffic is redirected to HTTPS at the CloudFront level rather than at the ALB.

In the demo environment, EC2 runs on t3.micro rather than the t3.medium specified in the design, and RDS runs Single-AZ on db.t3.micro rather than Multi-AZ on db.t3.small — both due to free tier account limitations. The architectural intent (Multi-AZ, private subnets, no public IPs, SSM access) is demonstrated regardless of instance size.

CloudCorp-ASG

CloudCorp-ASG Capacity overview [Edit](#)

arn:aws:autoscaling:eu-north-1:811261690901:autoScalingGroup:a73fa8b5-afbd-436f-81da-74fa8283d9e2:autoScalingGroupName/CloudCorp-ASG

Desired capacity 2	Scaling limits 2 - 4	Desired capacity type Units (number of instances)	Status Updating capacity
------------------------------	--------------------------------	---	--

Date created
Thu May 07 2026 11:48:45 GMT+0200 (centraleuropeisk sommartid)

[Details](#) | [Integrations](#) | [Automatic scaling](#) | [Instance management](#) | [Instance refresh](#) | [Activity](#) | [Monitoring](#) | [Tags](#)

Launch template [Edit](#)

Launch template lt-017c0efc8234211fb CloudCorp-LT	AMI ID ami-077d1b9f9a1902bbc	Instance type t3.micro	Owner arn:aws:iam::811261690901:user/AdminSempai
Version 2	Security groups -	Security group IDs sg-053eb97e30c9e61b6	Create time Thu May 07 2026 11:43:52 GMT+0200 (centraleuropeisk sommartid)
Description -	Storage (volumes) -	Key pair name -	Request Spot instances No

[View details in the launch template console](#)

Network [Edit](#)

Availability Zones eu-n1-az1 (eu-north-1a) eu-n1-az2 (eu-north-1b)	Subnet ID subnet-08c6429ac2b689080 subnet-00fc97b6d90fa22de	Availability Zone distribution Balanced best effort
---	--	---

Instance type requirements [Edit](#)

Your Auto Scaling group adheres to the launch template for purchase option and instance type.

Health checks [Edit](#)

Health check type EC2, ELB	Health check grace period 300
--------------------------------------	---

Daniel Hållbro @ CS25 IT- and Cybersecurity Specialist
Security analysis and redesign of CloudCorp's cloud architecture
Cloud and Virtual Service Security (30p)

Successfully created CloudCorp-DB-SubnetGroup. [View subnet group](#)

cloudcorp-db-subnetgroup

Subnet group details

VPC ID

[vpc-0197e2b0fc256b3c9](#)

ARN

arn:aws:rds:eu-north-1:811261690901:subgrp:cloudcorp-db-subnetgroup

Supported network types

IPv4

Description

Private data subnets for RDS

Subnets (2)

Availability zone	Subnet name	Subnet ID	CIDR block
eu-north-1a	AZ-1a-private-data	subnet-095f6a4e49dc48a90	10.0.3.0/24
eu-north-1b	AZ-1b-private-data	subnet-0239145a73fb1c557	10.0.6.0/24

Tags (0)

Filter by Tag key

Tags

You don't have any tags associated with this resource.

Manage tags

3.6 Edge and perimeter

AWS WAF is attached to the CloudFront distribution with four managed rule groups: Core Rule Set, Known Bad Inputs, SQL Database, and Amazon IP Reputation List. All four blocks by default.

WAF & Shield > Protection packs (web ACLs) > CloudCorp-WAF

resources and protection packs (web ACLs)

View your secured assets and their active security controls.

How it works

Protect your resources with packs
Protection packs are the new location for your web ACLs. Here you can supercharge your protections.

Watch live traffic
Use the graphs and dashboard to see how your resources are being affected. See your protections safeguard your resources in real time.

Smart recommendations
Based on traffic potential vulnerability, and aggregate data, we will give you tailor made recommendations to stop malicious actors before they attack.

Protection packs (web ACLs) (2)

Use protection packs (web ACLs) to combine rules and rule sets that protect your infrastructure from threats.

Search protection packs (web ACLs) by

Region scope

CloudFront (Global) and Regional

< 1 >

List

Grid

Name	Rules	Resources	Dashboard	AI traffic analysis	Sampled requ...	Logging & metrics
CloudCorp-WAF	4 rules	Manage	View	View	View	Not enabled

Manage rules for CloudCorp-WAF

< CloudCorp-WAF

Rules are shown in ascending rule priority order.

AWS-AWSManagedRulesCommonRuleSet

700 WCU

>

AWS-AWSManagedRulesKnownBadInputsRuleSet

200 WCU

>

AWS-AWSManagedRulesSQLRuleSet

200 WCU

>

AWS-AWSManagedRulesAmazonIpReputationList

25 WCU

>

Saved

Add rule

Cancel

Edit rule order

18

Review and create

General configuration

Distribution name
CloudCorp-CloudFront

Description
-

Billing
Pay-as-you-go (\$0/month)

Edit

Origin

Elastic Load Balancing origin
CloudCorp-ALB-480651081.eu-north-1.elb.amazonaws.com

Origin path
-

Grant CloudFront access to origin
No

Enable Origin Shield
No

Connection attempts
3

Connection timeout
10

Edit

Cache settings

Viewer protocol policy
Redirect HTTP to HTTPS

Allowed HTTP methods
GET, HEAD, OPTIONS, PUT, POST, PATCH, DELETE

Cache policy
[CachingDisabled](#)

Origin request policy
-

Response headers policy
-

Edit

Security

Security protections
[CloudCorp-WAF \(WAFv2\)](#)

Use monitor mode
No

Use existing WAF configuration
Yes

Edit

Cancel

Previous

Create distribution

3.7 Observability

CloudTrail is enabled as a multi-region trail logging all management events to **cloudcorp-data/logs/cloudtrail/** with log file validation enabled. Two CloudWatch alarms cover the most critical metrics: EC2 CPU above 80% and RDS CPU above 80%.

GuardDuty could not be enabled in the demo environment due to free tier account limitations. In production it would be enabled as a single-click operation and would immediately begin analyzing CloudTrail events, VPC Flow Logs and DNS logs for anomalous behaviour — directly addressing the total lack of threat detection that existed in the original environment.

Daniel Hållbro @ CS25 IT- and Cybersecurity Specialist
Security analysis and redesign of CloudCorp's cloud architecture
Cloud and Virtual Service Security (30p)

Choose trail attributes

Step 2

Choose log events

Step 3

Review and create

Review and create

Step 1: Choose trail attributes

Edit

General details

Trail name

CloudCorp-Trail

Multi-region trail

Yes

Apply trail to my organization

Not enabled

Trail log location

cloudcorp-data-811261690901/logs/cloudtrail/
AWSLogs/811261690901

Log file SSE-KMS encryption

Enabled

AWS KMS key

arn:aws:kms:eu-north-1:811261690901:key/bfc4857e-
ca05-4d76-8357-21b7fb76a845

AWS KMS key alias

CloudCorp-KMS-Stockholm

Log file validation

Enabled

SNS notification delivery

Disabled

CloudWatch Logs

Log group

cloudcorp-cloudtrail-logs

IAM Role

CloudCorp-Cloudtrail-Role

Tags

Key

Value

No tags

No tags associated with this trail

Step 2: Choose log events

Edit

Management events

No additional charges apply to log management events on this trail because this is your first copy of management events.

API activity

All

Exclude AWS KMS events

No

Exclude Amazon RDS Data API events

No

Data events

Data event collection is not configured for this trail

Insights events

Management events Insights types

API call rate

Enabled

Network activity events

Network activity event collection is not configured for this trail

Successfully created alarm CloudCorp-Alarm-RDS-HighCPU. View alarm

Alarms Mute rules - new

Alarms (2)

Filter alarms Quick filters

Clear selection

Create composite alarm

Actions

Create alarm

☐	Name	State	Actions	Actions muted - new	Last state update (UTC)	Conditions
☐	CloudCorp-Alarm-RDS-HighCPU	Insufficient data	No actions	-	2026-05-07 11:16:05	CPUUtilization > 80 for 1 datapoint: minute
☐	CloudCorp-Alarm-EC2-HighCPU	OK	No actions	-	2026-05-07 11:12:38	CPUUtilization > 80 for 1 datapoint: minutes

Part 4 — Network and secure design

This section dives into the network layer specifically. Part 2 covered the overall architecture and the principles behind it — here the focus is on how subnets, routing tables and security groups actually enforce the segmentation in practice.

4.1 Subnets and routing

The VPC has six subnets across two AZs, three per AZ: one public, one private app, one private data. The difference between public and private isn't the IP range — it's what the **route table** says.

Public subnet route tables have a default route (0.0.0.0/0) to the **Internet Gateway**. Anything here is directly reachable from the internet if a Security Group allows it, which is exactly why only components that *must* live here do: the ALB (must accept inbound HTTPS) and the NAT Gateways (must reach the internet for outbound traffic from private subnets). Putting an EC2 here, like in the original architecture, is what made the breach possible.

Private app subnet route tables point their default route to the **NAT Gateway** instead. EC2 instances can initiate stateful outbound traffic (OS updates, external APIs), but no inbound traffic from the internet can reach them outside the stateful window. This is the right place for application servers — they need to *make* calls outward, but should never *accept* calls from arbitrary internet sources.

Private data subnet route tables have no default route at all. RDS can only talk to other resources inside the VPC, and nothing on the internet can reach it directly. This is the strictest isolation level in this design, appropriate for the data layer where confidentiality is non-negotiable.

S3 traffic from EC2 is routed through a **VPC Gateway Endpoint** instead of the NAT Gateway, keeping S3 traffic on the AWS private network — this eliminates an exfiltration path *and* avoids NAT processing fees.

4.2 Security Groups

Security Groups act as instance-level firewalls. The design follows a chain pattern, where each component only accepts traffic from the component directly in front of it. Implementation details are in 3.2; this section motivates *why* the chain looks the way it does and what it actually achieves.

The chain — internet → ALB SG → EC2 SG → RDS SG. Each link is more restrictive than the one before it:

- **ALB SG** is the only one that accepts traffic from the internet, on 443 and 80 (the latter only to redirect to HTTPS). Outbound only forwards to EC2 on 443.
- **EC2 SG** does not accept traffic from the internet at all. Inbound is only 443 from the ALB SG. There is no SSH rule — admin access goes through Session Manager, which requires zero open ports.
- **RDS SG** is the most restrictive. Inbound only on 3306 from the EC2 SG. No outbound rule needed — RDS doesn't initiate connections.

The point of the chain is that an attacker who somehow got past the ALB still cannot reach the database directly. They would have to first compromise an EC2 instance, and even then only on port 3306. Everything else is blocked at the network level before authentication or application logic ever gets involved.

Two principles make the chain work in practice. First, Security Groups reference *other Security Groups by ID, not by IP*. The EC2 SG says "accept traffic from anything tagged ALB SG", not "accept traffic from a specific IP". When the ALB scales or moves IPs, the rule still works. When a new EC2 instance is launched by Auto Scaling, it inherits the SG and is automatically allowed. The rules describe *roles*, not *machines* — and roles are stable while machines come and go.

Second, **outbound rules are not optional**. AWS defaults to "allow all outbound", which makes life easy for an attacker who has already compromised an instance — exfiltration and additional malware downloads use exactly that path. Restricting outbound to what the application actually needs (HTTPS to AWS endpoints, MySQL to RDS, that's basically it) means a compromised EC2 has very little it can do once inside. The same thinking explains why no port 22 is open anywhere, not even from an admin's home IP: Session Manager makes SSH unnecessary, and removing the option closes a category of mistakes that "just for now" rules are notorious for becoming permanent.

4.3 Attack surface

An attack surface is what an attacker can reach from the internet without first compromising something. It's the surface area of "things that can be poked at remotely". The bigger it is, the more chances an attacker has — and the original architecture had it maximal.

Original architecture: the attack surface was effectively the entire environment. EC2 had a public IP with SSH (port 22) and HTTP (port 80) open to 0.0.0.0/0 — every botnet on the internet could attempt to brute-force credentials. The S3 bucket was publicly readable, indexable by search engines, and discoverable by automated scanners. Everything customer data touched was directly reachable from the internet.

New architecture: the only things reachable from the internet are CloudFront (hardened by AWS, DDoS-protected by Shield Standard, fronted by WAF) and the ALB it forwards to. Everything else — EC2, RDS, S3, the entire data layer — sits in private subnets without internet routes. EC2 has no public IP. There is no SSH port open anywhere; administrative access goes through Session Manager, which initiates outbound from the instance and requires zero inbound rules.

Even *if* an attacker compromised the ALB, the next hop is the EC2 Security Group — which only accepts traffic from the ALB itself, on port 443. From EC2 to RDS is another step gated by another Security Group, on a single port, from a specific source. This is the *defence in depth* pattern from 2.2 expressed as a graph: each node only accepts traffic from the node directly in front of it, and reaching the next node requires fully compromising the current one.

The remaining attack surface — CloudFront, WAF, ALB — is what AWS itself maintains and patches. That's near the practical minimum for an internet-exposed application that still needs to be reachable.

Part 5 — Security analysis

Even with the new design in place, things can still go wrong. The architecture in Parts 2 and 4 is much safer than what CloudCorp had before, but no setup is risk-free. This section goes through five risks that remain, with a short note on what can be done about each one, and ends with a broader improvement roadmap in 5.3.

5.1 Approach

The risks below come from the same threat model as the original incident: data leaks, misconfigurations, and misuse of legitimate access. Each one is realistic enough that it could affect CloudCorp within a year — not worst-case scenarios. There's no formal severity scoring. Instead each risk gets a short description of what could go wrong, why the current design doesn't fully prevent it, and what would.

5.2 Five identified risks

5.2.1 Operational expertise gap

The scenario is explicit that CloudCorp lacks deep security expertise. The new architecture introduces a number of services that need to be understood and operated correctly — GuardDuty findings, CloudTrail anomalies, CloudWatch alarms, IAM policies. Technical controls are only as good as the people interpreting their output. A WAF rule that fires on every login attempt is useless if nobody reads the alerts. This is the most important risk on the list because it cuts across all the others.

Mitigation: invest in AWS expertise, either by hiring a cloud security engineer or by funding AWS certifications for existing staff. This is an organisational measure, not a technical one, but the entire design depends on it.

5.2.2 Single-region dependency

The architecture is Multi-AZ within Stockholm, but everything still runs in one AWS region. A regional outage — rare but documented historically — would take CloudCorp completely offline. S3 Cross-Region Replication protects the data, but the application itself doesn't fail over. Recovery would require manually rebuilding compute and database resources in another region, which during an actual incident is not something CloudCorp's current team would be able to do quickly.

Mitigation: multi-region active-passive failover with Route 53 health checks. This was deferred in Part 2 as a future improvement because it roughly doubles infrastructure cost. As CloudCorp grows, the trade-off shifts and this becomes worth implementing.

5.2.3 Misconfiguration risk

The original incident wasn't a clever attack — it was a public S3 bucket. Misconfigurations remain the most common source of cloud breaches industry-wide. Block Public Access at the account level prevents the specific scenario that hit CloudCorp, but new misconfigurations can always emerge: an overly permissive IAM policy, a Security Group rule opened "just for testing", a missing KMS key on a new bucket. Manual configuration changes through the AWS Console are the highest-risk category.

Mitigation: automated checks that flag unsafe configurations before they cause damage, account-level rules that block the most dangerous changes from being made at all, and Infrastructure as Code (Terraform or CloudFormation) so changes go through code review rather than ad-hoc clicking.

5.2.4 Backup integrity and untested restoration

The architecture has good backup primitives in place — RDS automated backups, S3 versioning with MFA Delete, Cross-Region Replication for the main bucket. But having backups is not the same as being able to restore from them. Many organisations only discover at the worst possible moment that their backup files are corrupted, that the restoration process takes far longer than expected, or that critical data wasn't actually included in the backup scope. CloudCorp has no operational history yet to know how restoration would actually go.

Mitigation: periodic restoration drills where part of the database is restored from backup to a separate environment to verify that the process works end-to-end. Document expected recovery time and recovery point objectives (RTO/RPO) so the team has a baseline to measure against in a real incident.

5.2.5 Compromised credentials and insider threats

Technical controls protect poorly against legitimate credentials in the wrong hands. An IAM user whose access key is phished, a developer laptop compromised with active AWS sessions, or an angry employee with database access all bypass the network and Security Group controls completely. The original incident was caused by misconfiguration rather than a stolen credential, but the same outcome (data exfiltration) is reachable through credential abuse in the new design too.

Mitigation: mandatory MFA for all human IAM users (the simplest and most impactful step), reviewing GuardDuty findings regularly since they specifically flag anomalous IAM behaviour, and reducing reliance on long-lived access keys where possible — the goal is fewer credentials lying around to begin with.

5.3 Improvement roadmap

The five risks above point to a layered improvement plan rather than a single fix. Some of it is organisational, some technical, and the order matters.

Short term (next 1-3 months):

- **Build AWS expertise.** Hire or train. Without this, the rest of the roadmap can't be executed reliably.
- **Enforce MFA** for all human IAM access. Free, fast, immediate impact.
- **Turn on automated configuration checks.** AWS has built-in tooling that flags risky settings (like a publicly exposed bucket) automatically — useful for catching mistakes before someone else does.

Medium term (3-12 months):

- **Move infrastructure to code** (Terraform). Makes misconfigurations visible in pull requests rather than discovered after the fact.
- **Run a backup restoration drill.** Verify that backups actually work before needing them in an incident, and document RTO/RPO baselines.
- **Establish baseline** for CloudWatch alarms — tune thresholds based on observed traffic to reduce false positives.
- **Re-evaluate sizing decisions** once we have real usage data. A few of the choices in this design (EC2 t3.medium, RDS db.t3.small, ASG max 4) were set on the higher side on purpose, since we had nothing to base them on. After 3-6 months of CloudWatch data, we can adjust based on what's actually being used.

Long term (12+ months):

- **Multi-region failover** for true regional resilience. This is the biggest investment but also the one that meaningfully changes CloudCorp's recovery profile.
- **Periodic external pentest** or red team exercise to validate the architecture against active attackers, not just compliance checklists.

Security is not a project that finishes. It's a posture that has to be maintained as the threat landscape evolves and the application grows. The architecture in this report is a starting point that's significantly stronger than what came before — but the work doesn't end at deployment.

Part 6 — Cost and cloud service models

This section explains the cloud service models, where AWS costs actually come from, what cost risks the new architecture introduces, and how those risks can be mitigated. Verified cost figures for every service in the design are documented in Appendix A.

6.1 IaaS, PaaS and SaaS

The three service models differ in how much of the stack the provider manages versus the customer:

- **IaaS** — raw building blocks (VMs, storage, networking). The customer picks the OS, patches it, configures the firewall. AWS EC2, VPC and S3 are IaaS. AWS owns the hardware and hypervisor; everything above is the customer's job.
- **PaaS** — the customer brings application code, the provider handles the servers running it. AWS Elastic Beanstalk, Lambda and RDS sit here. Patching and scaling are managed; configuration and code are not.
- **SaaS** — the finished product, accessed through a browser or API. Gmail, Salesforce, and CloudCorp's own service.

CloudCorp **sells** SaaS but **consumes** IaaS to deliver it. That asymmetry matters: CloudCorp's customers pay not to think about EC2 or Security Groups, but CloudCorp itself very much has to. AWS only secures the cloud itself, not what runs inside it — the Shared Responsibility Model from the introduction is where this lives in practice.

6.2 Where the costs come from

AWS costs come from a few different dimensions that combine in sometimes unexpected ways. The four main categories are:

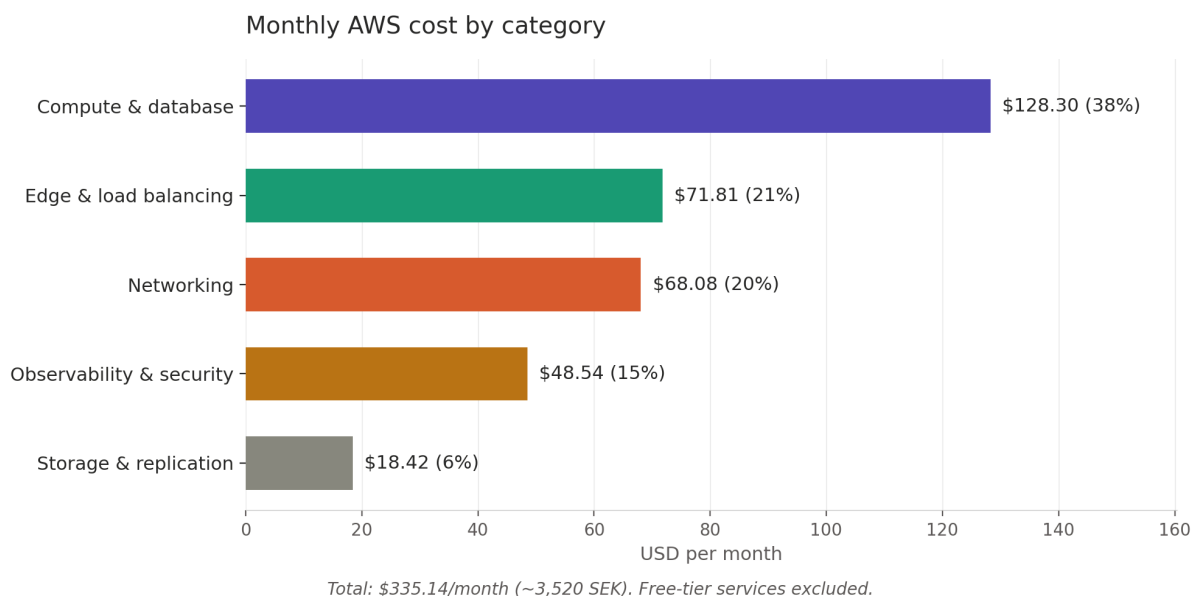
Compute time — EC2 instances, RDS instances, NAT Gateways and ALBs are charged per hour they exist, regardless of how busy they are. In our architecture this category alone makes up roughly \$220/month — both NAT Gateways together cost \$68/month even at zero traffic.

Data transfer — moving data between AWS and the internet costs money, and so does moving data between regions. Moving data within the same region is mostly free, with the important exception of NAT Gateway processing fees. CloudFront has its own pricing model where data leaving the edge to users is the most expensive component.

Requests and operations — most managed services charge a small amount per API call or operation. S3 charges per PUT and GET, CloudTrail charges per data event, WAF charges per inspected request. The numbers are tiny per call, but at high request volumes they become a meaningful cost driver.

Storage — costs are calculated per GB/month, and storage classes vary by retrieval cost. S3 Standard is fast but more expensive per GB than Glacier, which is cheaper but slow to retrieve.

The new architecture lands at roughly \$335/month for the size of CloudCorp described in the scenario. The largest single posts are NAT Gateway (\$68 — fixed cost for Multi-AZ redundancy), EC2 (\$64), and RDS (\$64 — Multi-AZ doubles the instance cost). Detailed figures for every service are in Appendix A.



6.3 Cost risks

Cloud costs can spiral for reasons that have nothing to do with traffic growth. Four risks specific to this architecture are worth flagging.

The first is **NAT Gateway data processing fees**. NAT charges \$0.046 per GB processed in addition to the hourly fee. In the current design this is a non-issue — S3 traffic goes through the VPC Gateway Endpoint and the application's outbound traffic is limited to OS updates and external API calls. The risk shows up later, as the architecture

evolves. Adding a third-party integration that ships large volumes of data, a future log forwarder to an external service, or even container image downloads from an external registry can quickly increase NAT volume. As an illustrative example, an integration that pushes 100 GB/day outbound through NAT would add about \$138/month in processing fees alone — almost as much as the rest of the NAT bill combined. The mitigation is the same pattern we already use: a VPC Endpoint for any AWS service that becomes a major traffic source.

The second risk is the **free tier cliff**. CloudFront includes 1 TB data transfer and 10 million requests per month for free, after which pricing kicks in. CloudCorp's modeled traffic of 1.2 TB and 15M requests/month sits just above those thresholds — the first 1 TB and 10M are still free, but the 200 GB and 5M that exceed them are what generates the \$24/month bill. The risk isn't the cliff itself (we're past it) but how steep the slope becomes if traffic grows. Going from 1.2 TB to 2 TB would add roughly \$68/month in transfer alone, doubling the CloudFront bill for what feels like only a small traffic increase. Without monitoring, the first sign of this is the next month's invoice. AWS Budgets alarms are configured precisely to catch that kind of jump.

The third risk is **default settings that are not cost-aware**. Detailed monitoring on EC2, Performance Insights with extended retention on RDS, GuardDuty Malware Protection — all of these are easy to enable through a few clicks and all of them add cost. The risk isn't that any single one is expensive (most are a few dollars per month), but that they accumulate silently across services. Reviewing the AWS bill monthly is the only reliable way to catch this drift before it becomes meaningful.

A fourth risk is **silent cost drift from things that go unnoticed**. A developer spins up an EC2 instance for debugging and forgets to shut it down. Application logs grow into gigabytes per day after a verbose debug flag was left enabled in production. A test RDS instance from a migration is never deleted. None of these are dramatic individually, but they add up between invoices. AWS Budgets and Cost Anomaly Detection exist for exactly this — automatic alerts when spending deviates from the established baseline.

6.4 Cost optimisations

Several optimisations are already baked into the design. Others are deferred until CloudCorp has enough operational history to make informed decisions.

Already in place:

- **VPC Gateway Endpoint for S3** keeps S3 traffic off the NAT Gateway, eliminating both an exfiltration path and a recurring data charge
- **VPC Flow Logs sent to S3 instead of CloudWatch Logs** — both destinations store the same data, but S3 ingestion is roughly half the price (\$0.271/GB vs \$0.54/GB in Stockholm). VPC Flow Logs are large in volume but rarely queried

in real-time, which makes the cheaper destination the right fit.

- **Limit of 3 dashboards** keeps CloudWatch dashboards within the free tier
- **MFA Delete on the logs and backups bucket** prevents an attacker from silently destroying forensic evidence (which would otherwise force expensive incident response from scratch)

Deferred until observation:

- **3-year Compute Savings Plans on EC2 baseline** would reduce EC2 cost from \$63 to ~\$36/month — a 42% saving. This is deferred because committing to instance types for three years requires confidence in the workload, which CloudCorp doesn't have yet
- **S3 lifecycle policies** that move old logs and backups to cheaper storage tiers — recent data stays in Standard (\$0.023/GB), while data older than 30 days moves to Standard-IA (~half the price) and data older than a year moves to Glacier (a fraction of that). For an organisation that accumulates logs and backups over time, this typically reduces storage cost by 30-40% after the first year, automatically.
- **Right-sizing EC2 to t3.small after observation** — t3.medium was chosen at deployment time to provide headroom for unknown actual usage in a multi-tenant environment. After 3-6 months of CloudWatch metrics showing sustained CPU and memory below 50%, downsizing to t3.small would save ~\$32/month while preserving Auto Scaling as the safety net for occasional spikes. The same logic applies to RDS — if db.t3.small turns out to be more than enough, downsizing further would save additional money

The most important thing about cost optimization is that **it should not come before security**. Several savings — removing Multi-AZ (\$60/month), going to a single NAT Gateway (\$34/month) — defeat the point of the redesign. The cost in this architecture is what it needs to be for the security posture it provides.

Reflection

No architecture is finished the day it ships. The one I designed is much stronger than what CloudCorp had before, but there are still risks I couldn't fully solve, and the controls in place will need attention as both the threat landscape and the application keep evolving.

The other thing I kept coming back to was trade-offs. There's almost always a more expensive, more redundant, or more complex option available — but more isn't automatically better. I tried to be honest about why I chose what I chose, including where I went conservative on purpose (like keeping EC2 on t3.medium instead of right-sizing too early) and where I cut back deliberately (like skipping multi-region disaster recovery for now). Going through the reasoning for each choice — including the ones I almost talked myself out of — made me actually think about why I picked what I picked.

Finally, working with real numbers from the AWS Pricing Calculator instead of rough estimates changed how I thought about the design. It's easy to write "this is justifiable for production" without doing the math, but as soon as the cost is concrete (~\$335/month, ~3,520 SEK), every choice gets weighed against what it actually costs versus what it actually protects. Looking things up instead of guessing is a habit worth keeping.

If I started this over, I would build the environment as **Infrastructure as Code (IaC)** rather than clicking through the console. Doing it manually was useful for learning but **ClickOps** produces a fragile result with no single source of truth and no diff if something drifts. There is a real irony in writing a security report about a misconfiguration breach using a method that is itself a leading cause of such breaches. In production, treating infrastructure as code is one of the controls. If CloudCorp were to start using **Compliance as Code**, they need to be using **IaC** in order for it to work.